

Subject: Operating System

Unit : 4

Prof. Ashwani Mishra

Assistant Professor
AIML Department

CONTENTS

- Memory Management
- Contiguous Memory Allocation and Segmentation
- Paging and Segmentation
- Virtual Memory

Memory Management

- The programs, together with the data they access, must be in the main memory (at least partially) during execution. To improve both the utilisation of the CPU and the speed of its response to users, the computer must keep several processes in memory.
- Many Memory-management schemes exist, reflecting various approaches, and the effectiveness of each algorithm depends on the situation. Memory is central to the operation of a computer system. Memory consists of a large array of words or bytes, each with its own address.
- The execution of a program will fetches instructions from the Memory. After decoding of an instruction, the CPU again fetches memory for operands. Once the execution is completed, then results are stored in memory.

Dynamic loading

- The process execution must need the entire program and data in physical memory. In this case, the size of the process is limited to size of physical memory.
- The dynamic binding is remedy for this limitation. In the dynamic loading, all the routines are kept on disk in a relocatable load format.
- The routine is loaded when it is called by the process.

Continued...

Dynamic linking and shared libraries

- If the linkage of libraries are performed at execution time, then it is called dynamic linking. The dynamic linking makes use of stub, where the stub is a small piece of code that indicates how to locate the appropriate memory-resident library routine or how to load the library if the routine is not already present.
- This process is used for library updates. A library may be replaced by new version then all the program that are referring this will automatically use the new version. This system is known as shared libraries.
- Overlays will enable a process to be larger than the amount of memory allocated to it.

Swapping

- A process is swapped temporarily out of memory to back store, and brought back into memory to continue the execution.
- A multi-programming system with round-robin scheduling will swap out the process when time quantum is expired; upon their round, it will be brought back into the memory for execution. This process is called swapping.
- The swapping is even used in priority-based systems also. The swapping is sometimes called as roll out, roll in. For compile time and load time bindings, the roll-out process will occupy the same memory addresses after the roll-in.

Contiguous Memory Allocation and Segmentation

The memory is usually divided into two partitions:

- Operating system: It is possible to place the operating system in either low memory or high memory. The major factor of affecting this decision is the location of the interrupt vector. Since the interrupt vector is often in low memory, it is more common to place the OS in low memory.
- User processes, where we want to reside it in memory at the same time.
- In Contiguous memory allocation, each process is contained in a single contiguous section of memory.

Memory Allocation:

- One of the simplest method for memory allocation is to divide memory into several fixed-sized partitions, where each partition may contain exactly one process. This method is used by the IBM OS / 360 Operating system (called MFT); it is no longer in use.
- The method described next is a generalisation of the fixed-partition scheme (called MVT); it is used primarily in a batch environment.

Contiguous Memory Allocation and Segmentation

Fragmentation:

- Memory fragmentation can be internal or external to the address space of a process.
- The general approach is to break the physical memory into fixed-sized blocks, and allocate memory in unit of block sizes. By the above approach, the memory allocated to a process may be slightly larger than the requested memory.

Continued...

Segmentation

- The user's view of memory is not the same as the actual physical memory.
- The user's view is mapped onto physical memory.
- The mapping allows differentiation between logical memory and physical memory.

Basic Method

- Each of the segments is of variable length; the length is intrinsically defined by the purpose of the segment in the program. Elements within a segment are identified by their offset from the beginning of the segment: the first statement of the program, the tenth entry in the symbol table, and so on.

Segmentation

- Segmentation is a memory -management that supports this user view of memory. A logical address space is a collection of segments. Each segment has a name and a length.
- The addresses specify both the segment name and the offset within the segment.
- For simplicity of implementation, segments are numbered and are referred to by a segment number, rather than by a segment name.
- Logical address consists of a two tuple:
- < segment-number, offset >

Continued...

A Pascal compiler might create separate segments for the following:

- The global variables
- The procedure call stack, to store parameters and return addresses, the code portion of each procedure or function.
- The local variables of each procedure and function. A Fortran compiler creates a separate segment for each common block. Arrays may be assigned separate segments.
- The loader would take all these segments and assign them segment numbers.

Protection and Sharing

- All entries in the segment are used in the same way, where some segments are instructions, and others are data. In modern architecture, instructions are non-self-modifying, so instruction segments can be defined as read-only or execute-only. The memory-mapping hardware will check the protection bits associated with each segment table entry to prevent illegal accesses to memory.
- Thus, many common program errors will be detected by the hardware before they can cause serious damage.
- Another advantage of segmentation involves the sharing of code or data.

Continued...

- Segmentation may cause external fragmentation, when all blocks of free memory are too small to accommodate a segment.
- In this case, the process have to wait until more memory becomes available, or until compaction creates a large hole because segmentation is a dynamic relocation algorithm.
- The amount of external fragmentation is proportional to the average size of the segment. Hence by keeping the segment size smaller we can control the loss due to external fragmentation.

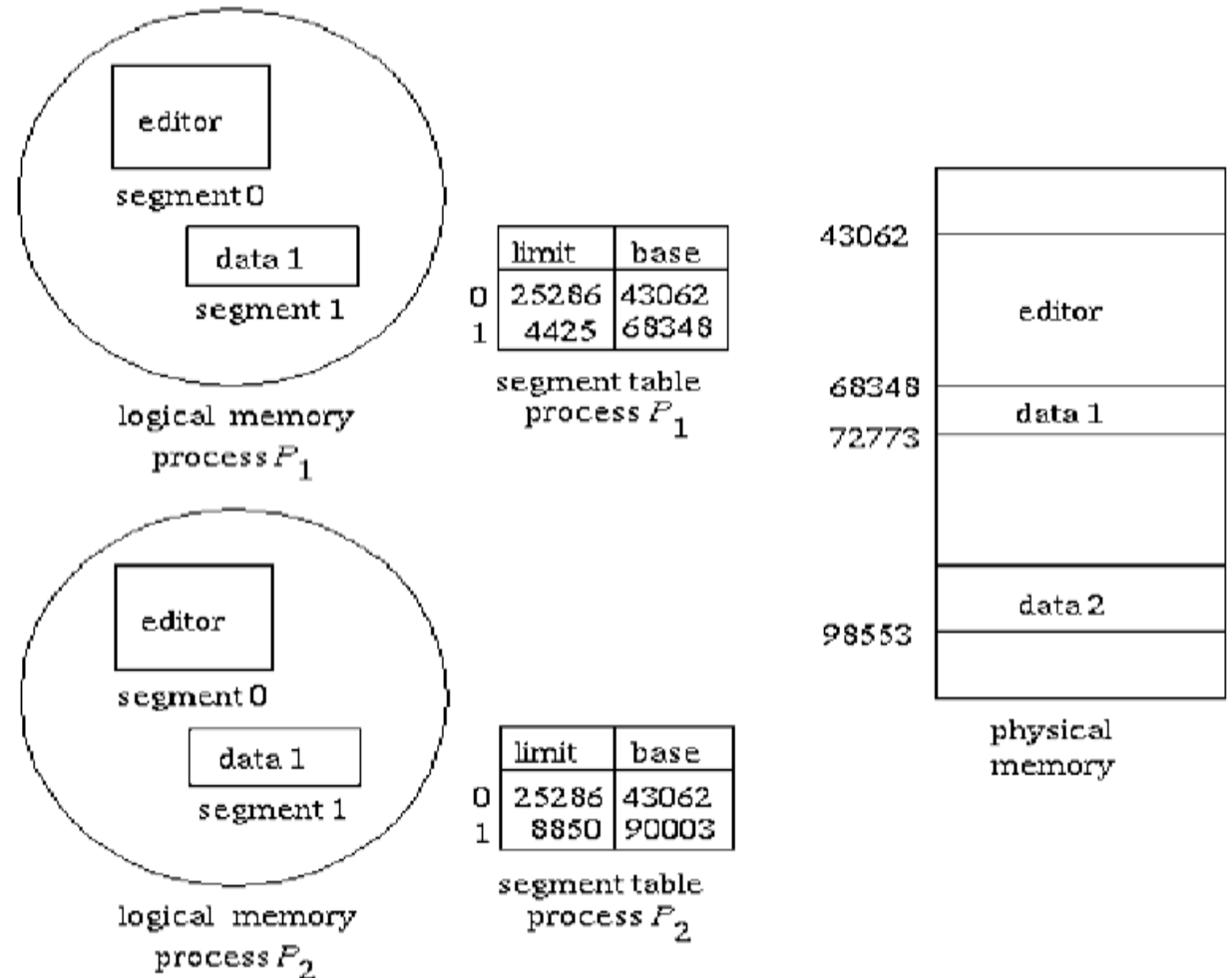


Figure : Example of segmentation

Paging & Segmentation

- Paging is a memory management scheme which permits the physical-address space of a process to be noncontiguous. Paging avoids the problem of fitting variable sized blocks to the backing stores. The paging concept will divide the physical memory into fixed sized blocks called frames and logical address space is divided into pages, but the page and frame size must be equal.

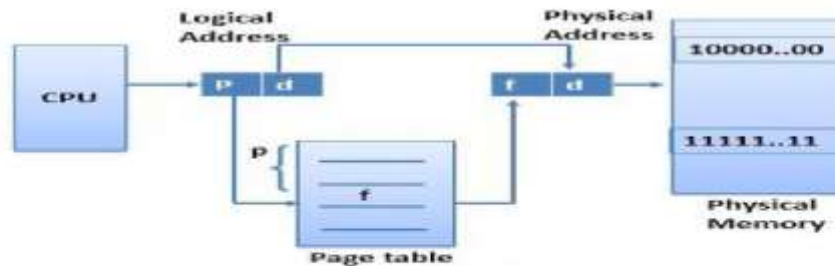


Figure: Paging Hardware

The paging will divide the CPU generated addresses into two parts: page number (p) and page offset (d). The page number is used as an index to the page



Figure: paging model of logical and physical memory

- The page table contains the base addresses of pages in physical memory and the OS provides it. The size of the page is defined by the hardware.
- The page size selection makes the translation of logical address into page number and page offset, and the page size is typically a power of 2. If the size of a logical address space is 2^m , and a page size is 2^n then the high-order $m-n$ bits of a logical address specifies the page number and lower order n bits are page offset.

Continued...

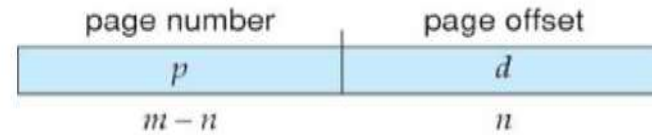


Figure: Logical address

- The paging is a form of dynamic relocation where every logical address is bound by the paging hardware into a physical address. The OS will support this conversion using a page tables. In addition to this, it also maintains a frame table in which the allocation details of physical memory such as which frames are allocated, which frames are free, and how many frames are there etc., are kept.

Implementation of Page Tables

- The implementation of page table is done in several ways. In the simplest case it is implemented as a set of dedicated registers. The use of registers for page table is efficient but it is satisfactory for small page tables. In other case the page table is kept in main memory and the page table base register points to the page table but this requires more access to user memory.

Continued...

Hardware Support:

The use of translation look-aside buffer (TLB) provides standard solution to this problem and TLB is small, fast lookup hardware cache. The entry of TLB contains two parts: a key and a value. If memory is presented with an item then it is compared with all keys and the value is returned if it is found. The TLB maintains only few of the page table entries.

Continued...

Segmentation of with Paging

- Paging scheme for memory management suffers from internal fragmentation, whereas the segmentation scheme suffers from external fragmentation.
- A hybrid scheme that combines paging and segmentation to improve on each is illustrated by the architecture of the Intel 386.
- The IBM OS/2 32-bit version is an operating system running on top of the Intel 386 architecture.
- The 386 uses segmentation with paging for memory management.
- Maximum number of segments per process is 16K. Each segment can be as large as 4 gigabytes.
- Page size is 4 KB.
- The logical address space of a process is divided into two partitions.
- Consists of up to 8KB segments that are private to the process Kept in the local descriptor table (LDT). And 8 KB segments that are shared among all the processes 'Kept in the global descriptor table (GDT)'.

Virtual Memory

- Virtual memory is a technique which allows the execution of processes that may not be completely in the memory. User program can be larger than physical memory. It abstracts main memory into extremely large uniform array of storage, separating logical memory as viewed by the user from physical memory.
- The Memory management schemes impose restrictions on the physical size of the memory, but in real case, the entire program is not needed. For example,
 - Programs often have code to handle unusual errors conditions the code almost never used.
 - Arrays, lists and tables are often allocated more memory than they are actually need.
 - Certain options and features of a program may be rarely used (text editor command to convert all the lower case letters into upper case).
- The ability to execute a program which is only partially in memory would have many benefits.
- A program would no longer be constrained by the amount of physical memory that is ' available.
- Since each user could take less physical memory, more users can be run at the same time, which increases CPU utilization and throughput.

Continued ...

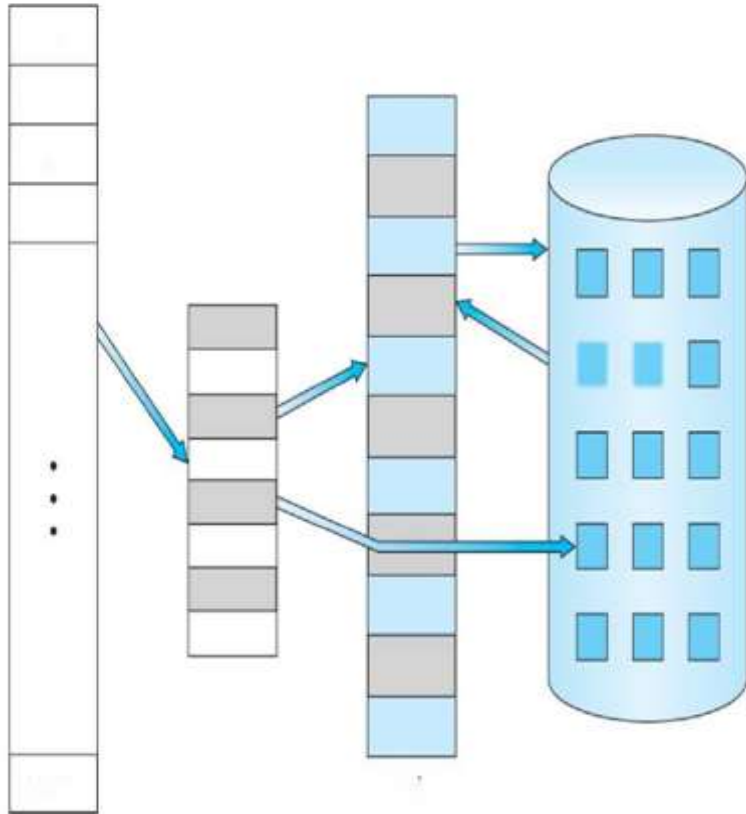
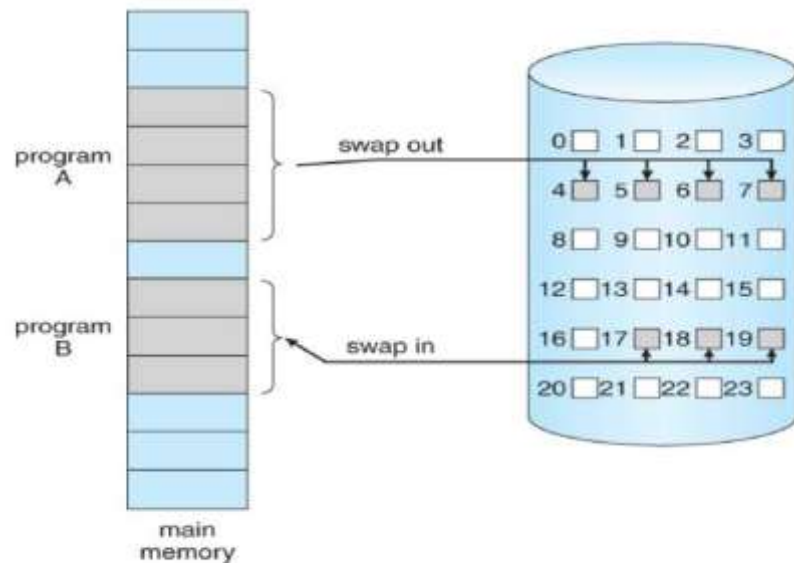


Figure : Showing Virtual Memory Larger than Physical Memory.

- A technique called overlays is used to allow a program to be larger than the amount of memory allocated to it. That is in those instructions and data that are needed at any time are kept in memory.
- Virtual memory is commonly implemented by demand paging, and it can also be implemented in a segmentation system. Several systems provide a paged segmentation scheme where segments are broken into pages. Thus, the user view is segmentation, but the OS can implement this view with demand paging.
- Demand Segmentation can also be used to provide virtual memory. However, segment replacement algorithms are complex than page replacement algorithms because the segments have variable sizes.
- Dynamic loading loads the routine when it is called. All the routines are kept on disk in a relocatable load format. The advantage of dynamic loading is that, unused routine is never loaded.

Demand Paging

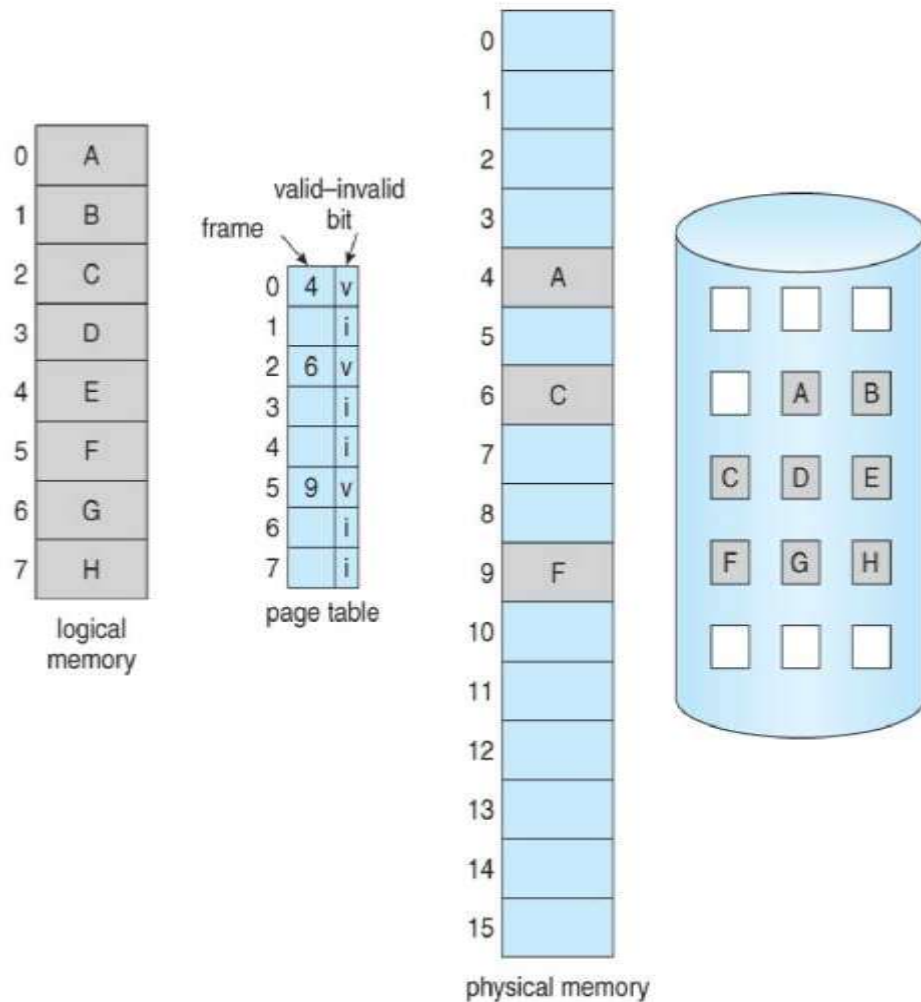
- A demand paging system is similar to a paging system with swapping. Processes reside on secondary memory (which is usually a disk). When want to execute a process, swap it into memory.
- The lazy Swapper never swaps a page into memory unless that page will be needed; rather than swapping the entire process into memory, we use lazy swapper. A swapper manipulates the entire processes, whereas a pager is concerned with the individual pages of a process. So, we use the term pager rather than swapper.



Transfer of paged Memory to contiguous disk space.

- When a process is to be swapped in, the pager guesses which pages will be used before the processes swapped out again. Thus it avoids reading into memory .pages that will not be used any way, decreasing the swap time and the amount of physical memory needed.
- To distinguish between pages that are in memory and those that are on disk, we require valid and invalid bit.
- Valid bit indicates the page is legal and is in memory.
- Invalid bit indicates whether the page is invalid or valid but is currently residing on the disk.

Continued...



Page table when some pages are not in Main Memory

- Access to a page marked invalid causes a page fault trap. The paging hardware, in translating the address through the page table, will notice whether the invalid bit is set or not, causing a trap to the OS.
- This trap is the result of the OS's failure to bring the desired page into memory, rather than an invalid address error as a result of an attempt to use an illegal memory address. The procedure for handling this page fault is simple.
- We check an internal table for this process to determine whether the reference was a valid or invalid memory access.
- If the reference was invalid, we terminate the process. If it is found valid, but we
- have not yet brought that page in we now page in.
- Find a frame. Schedule a disk operation to read the desired page into the newly allocated frame.

Performance of Demand Paging

- Demand paging can have a significant effect on the performance of a computer system. To see why, let us compute the effective access time for a demand-paged
- Memory.
- Let p be the probability of a page fault ($0 \leq p \leq 1$). We would expect p to be close to zero; that is, there will be only few page faults. Then Effective access time = $(1-p) * m_a + p * \text{page fault time}$.
- A page fault causes the following sequences: Trap to the OS.
- Save the user registers and process state.
- Determine that the interrupt was the page fault.
- Check that the page reference was legal and determine the location of the page on the disk.
- Issue the read from the disk to a free frame.
- While waiting, allocate the CPU to some other user.
- Interrupt from the disk.
- Save the registers and process state for the other user.
- Determine that the interrupt was from the disk.

Continued...

- Correct the page table and other tables and the other tables to show that the desired page is now in memory.
- Wait for the CPU to be allocated to this process again.
- Restore the user registers, process states, and new page table then resume the interrupted instruction.

In any case, we are faced with three major components of the page fault service time:

- Service the page fault interrupt
- Read in the page
- Restart the process.
- If we take an average page fault service time 25 milliseconds and a memory access time of 100 nanoseconds, then

$$\begin{aligned}\text{Effective access time in nanoseconds} &= (1-p) * (100) + p (25 \text{ milliseconds}) \\ &= (1-p) * (100) + p (25000000) \\ &= 100 + 24,999,900 * p.\end{aligned}$$

Thus the effective access time is directly proportional to the page fault rate. In one access out of 1000 causes a page fault, the effective access.

Page Replacement

Page Replacement

- It might happen that the process that is executing may find that the required page is not in memory and page fault occurs. The hardware traps to the operating system, to check its internal table it is a page fault and is a valid page, i.e., not an illegal memory access. The OS where the desired page is in the backing store but the finds there are no free frames on the free frame list, all of the memory is in use.
- The OS system could terminate the user program. If no frame is available for, find one which is not currently being used and free it. We can free a frame by writing its contents to the backing store and changing the page "table to indicate that the page no longer in memory. The freed frame can now be used to hold the page for which the program faulted.

Need for Page Replacement

- The page is searched on the disk. But if there are no free frames available in the memory the operating system can terminate the user process, swap out a process freeing its frame using a page replacement algorithm.
- Its page replacement takes the following approach. If no frame is free
- Find the location of the desired page on the disk.
- Find a free frame: If there is a frame, use it.
- Otherwise, use a page-replacement algorithm to select a victim frame.

THANK YOU